Assignment 6: Schelling's Model

For this assignment, you will implement Schelling's model of segregation.

Instructions

In Schelling's model, we start with a gride containing a randomly distributed population of 2 or more colors. In each iteration of the program, each non-empty cell is given a chance to move. If the cell has 2 or more neighbors of the same color, then it decides to stay where it is. Otherwise, the cell moves one step in any direction, preferably to the location with the most similar neighbors.

Start with your code from the previous "Colorful Static" assignment and implement the algorithm described above. If you need some help, here are my suggestions:

- In your `__init__` method:
  - Create a new member variable called `desired_similar_neighbors` and initialize it to 2.
  - Initialize your grid of cells with randomly assigned values of 0, 1, 2, or 3.
  - It may help to have extra empty cells. See my code below for a suggestion of how to do this.
- If your `draw` method is just drawing the cells with colors based on their values, then it doesn't need to change. (0 = no color, 1 = red, 2 = blue, 3 = green).
- Write a new `update` method:
  - If the cell's value is 0, do nothing; otherwise:
  - Count the number of adjacent neighbors with the same value.
  - If the number of similar neighbors is less than `desired_similar_neighbors`, then:
    - Examine the empty neighboring cells
    - Choose an empty cell to move to. (The choice is up to you. Random? First available? The one with the most similar neighbors?)
    - Set the new location to this cell's value.
    - Set the old cell location to 0.

Feel free to experiment with your code that chooses a new location. You can also experiment with changing `desired_similar_neighbors`. You can also increase or decrease the number of colors.

Here's some code to initialize the grid of cells with a given density.

```python
class Game:
    def __init__(self):
        # other initializations...

        self.num_cells_x = 256
        self.num_cells_y = 144
        self.num_groups = 3
        self.density = 0.3
        self.grid = []

        for _ in range(self.num_cells_y):
            row = []
            for _ in range(self.num_cells_x):
                if random.random() < self.density:
                    row.append(random.randrange(self.num_groups) + 1)
                else:
                    row.append(0)
            self.grid.append(row)
```

After some time, the output of your program should look something like this. Note how the cells have clumped together.